

1 RAV KOMBAT: CHOOSE YOUR DESTINY!



Figure 1: EDA Games - RAV Kombat: Choose your Destiny!

*¿Tienes lo suficiente para ser el supremo campeón de RAV Kombat o solamente eres patético? ¡Elige tu destino!*¹. La compañía EDA Games se prepara para el lanzamiento de su nuevo video-juego de peleas llamado RAV Kombat: Choose your Destiny!, en el cual un grupo de estudiantes de la UCM (Universidad de C Manía), han sido invitados a un torneo de artes marciales. Si alguno de estos estudiantes logra hacerse con el torneo, obtendrá el título de Campeón Supremo de RAV Kombat, junto con la potestad de utilizar una Rebaja Académica Voluntaria (RAV) extra durante el transcurso de su carrera.

Para lograr la victoria, el estudiante deberá atravesar una torre de desafío, donde irá combatiendo de forma secuencial con un grupo aleatorio de profesores de diversas organizaciones mundiales. Cuando derrote al último maestro, será coronado como Campeón Supremo de RAV Kombat.

1.1 Sistema de Personajes

Cada personaje en RAV Kombat, tanto estudiantes como profesores, cuentan con los siguientes atributos:

- **Nombre:** String que indica el nombre del personaje.
- **ataque:** Flotante que indica la cantidad de daño base del personaje.
- **defensa:** Flotante que indica la cantidad de defensa base del personaje.
- **vida:** Flotante que indica la cantidad de vida que tiene el personaje. Este valor debe ser multiplicado por 1000.

¹Basado en los audios y propagandas de los juegos de Mortal Kombat II y Mortal Kombat 3

Los estudiantes cuentan con el atributo exclusivo **suerte**, un factor que modifica la capacidad base de los atributos anteriormente explicados. Los profesores, por otra parte, cuentan con el atributo exclusivo **modo_dificultad**, un factor que aumenta sus atributos de acuerdo a su ubicación en la torre de desafío.

1.2 Sistema de Selección de Personaje y Torre de Desafío

Para poder iniciar un torneo, los jugadores tendrán que tomar ciertas decisiones:

1. Seleccionar un estudiante: De un roster de doce (12) icónicos estudiantes de la UCM elegidos para RAV Kombat, el jugador deberá seleccionar sólo a un (1) participante para el torneo.
2. Seleccionar una Torre de desafío: De un total de 3 dificultades; fácil, normal y difícil; el jugador deberá escoger una torre de desafío para completar el torneo. Éstas tienen las siguientes características:
 - Torre Fácil: Cuenta con batallas contra 10 profesores; siete (7) de nivel fácil y tres (3) de nivel normal.
 - Torre Normal: Cuenta con batallas contra 10 profesores; cinco (5) de nivel fácil, cuatro (4) de nivel normal y uno (1) de nivel difícil.
 - Torre Difícil: Cuenta con batallas contra 15 profesores; cinco (5) de nivel fácil, cinco (5) de nivel normal y cinco (5) de nivel difícil.

El torneo comenzará con dificultad progresiva, desde el fondo de la torre de desafío; contra el profesor con menor dificultad; hasta llegar, uno por uno, al profesor de mayor dificultad. Considere que no existe diferencia de dificultad entre profesores del mismo nivel, a pesar de que sus atributos puedan variar.

1.3 Sistema de Peleas y Recompensas

Las peleas son combates entre un estudiante que ingresa a la torre de desafío y un profesor en cierto nivel de la torre. Para determinar quien vence, se sigue el siguiente proceso de golpes iterativos:

- Cada personaje sumará sus atributos de **ataque** y **vida**, luego lo multiplicará por el atributo de **suerte** o **modo_dificultad**, según corresponda, para finalmente restarle el atributo **defensa** del rival.
- Si el resultado del calculo anterior, es positivo, se debe ir restando a la vida del rival.
- El primer personaje que lleve su rival a cero (0) o negativo de **vida** en menos golpes, será el vencedor.
- Además, si el estudiante vence al profesor, tendrá la posibilidad de efectuar un movimiento de finalización llamado "**Hundredality!!!!**". Este movimiento tiene la probabilidad de ser efectuado *una vez en la cantidad de golpes que llevó a cabo* para vencer al profesor.

Si el estudiante sale victorioso, además será recompensado con un atributo de su oponente: ataque, defensa o vida. Esta recompensa será añadida al nivel de mejoras, un sistema que permite aumentar el respectivo atributo en cinco unidades (+5) cada vez que se añadan tres (3) recompensas de un atributo. Si el estudiante sale derrotado, tiene tres oportunidades de continuar, empezando desde la base de la torre de desafío, manteniendo todos los atributos adquiridos antes de ser derrotado.

1.4 Bienvenidos a EDA Games

EDA Games lo ha contratado como desarrollador de RAV Kombat: Choose your Destiny!, asignandolo como programador de prototipo en línea de comandos. Para esto, EDA Games le entrega lo siguiente:

- Un archivo cabecera llamado `ravk.h` el que incluye bibliotecas y estructuras a utilizar en el desarrollo del prototipo. **Considere que todo lo que está en la cabecera debe ser utilizado. Además, no podrá eliminar elementos allí definidos, sólo agregar cosas nuevas.**

`ravk.h`

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>

typedef struct nivel_de_mejoras{
    char * premio;
    struct nivel_de_mejoras * sgte;
}stats_t;

typedef struct estudiante{
    char * nombre; // nombre del estudiante
    float ataque;
    float defensa;
    float vida;
    float suerte; // ratio de suerte
    stats_t * stats_ataque; // nivel de mejoras: ataque
    stats_t * stats_defensa; // nivel de mejoras: defensa
    stats_t * stats_vida; // nivel de mejoras: vida
    struct estudiante * sgte;
}noob_t;

typedef struct profesor{
    char * nombre; // nombre del profesor
    float ataque;
    float defensa;
    float vida;
    float modo_dificultad; // ratio de dificultad
    char * premio; // string de recompensa
    struct profesor * sgte;
}profe_t;

typedef struct seleccion{
    noob_t *roster;
}char_t;

typedef struct torre{
    int nro_de_profes;
    profe_t * tower;
}tower_t;
```

- Un archivo llamado `profesores_npc.csv`, el cual contiene la información referente a los profesores que uno puede encontrar en el juego. Este archivo contiene la información necesaria para el juego, incluyendo: nombre, ataque, defensa, vida, nivel_desafio y el tipo de premio si es derrotado. Los puntos suspensivos indican que hay muchas más líneas en el archivo.

`profesores_npc.csv`

```
nombre;ataque;defensa;vida;nivel_desafio;premio
NiKoFuNk;10;5;4;dificil;ataque
God_Rahamon_Descartes;2;6;7;dificil;vida
Nico_towers;12;1;2;dificil;ataque
Marie_CircuiTikz;13;1;1;dificil;ataque
Duck_olives;9;1;5;medio;ataque
Patricio_guisante;1;2;12;facil;vida
...
default_progra_NPC_22;12;1;2;facil;ataque
Sr._Shepherd;9;1;5;medio;ataque
default_MAT203_NPC_1;2;9;4;medio;defensa
...
```

- Un archivo llamado `estudiantes_elegibles.csv`, el cual contiene la información referente a los estudiantes que el jugador podrá elegir para elegir alguna de las torres de desafío. Este archivo contiene la información necesaria para el juego, incluyendo: nombre, ataque, defensa, vida, y el nivel de suerte. Los puntos suspensivos indican que más líneas en el archivo.

`estudiantes_elegibles.csv`

```
nombre;ataque;defensa;vida;factor_suerte
Motita_master;7;3;5;0.99
Seba_ossas;12;1;2;1.15
omelo_chino;13;1;1;1.11
gibio;2;10;3;1.06
don_pollo;2;1;12;1.14
...
```

Considerando lo anteriormente entregado, **EDA Games** le solicita desarrollar lo siguiente:

- (10 puntos) Leer el archivo de configuración de los Estudiantes que han sido invitado al torneo RVA Kombat y cargarlos en una Estructura de Datos (EDA) de su elección, encargada de manejar eficientemente el sistema de selección de personaje. **Considere que el nombre del archivo de configuración inicial será obligatoriamente un parámetro del ejecutable.**
- (10 puntos) Leer el archivo de configuración de los Profesores al cual se puede enfrentar los Estudiantes en el torneo RVA Kombat y cargarlos en tres Estructura de Datos (EDA) independientes, según la dificultad de cada torre de desafío. La EDA debe ser la misma para las tres torres, pero debe ser diferente a la utilizada para selección de personaje. **Considere que el nombre del archivo de configuración inicial será obligatoriamente un parámetro del ejecutable.**
- (15 puntos) Crear un archivo, llamado `roster.c`, en donde implemente la interfaz de la EDA escogida para manejar la selección de personajes. Esta EDA debe usar obligatoriamente el `struct seleccion` o `char_t` como base. Considere que la interfaz debe ser capaz, al menos, de:
 - Añadir un nuevo estudiante a la EDA.
 - Buscar la ubicación de un estudiante en la EDA.
 - Seleccionar un estudiante de la EDA.
 - Imprimir la info de todos los elementos de la EDA.
 - Vaciar la EDA.
- (15 puntos) Crear un archivo, llamado `towers.c`, en donde implemente la interfaz de las Estructuras de Datos (EDA) escogidas para manejar la diferentes torres de desafío de RAV Kombat. Esta EDA debe obligatoriamente usar el `struct torre` o `tower_t`. Además, considere que la interfaz debe ser capaz, al menos, de:
 - Añadir un profesor en alguna de las EDAs.
 - Eliminar un profesor en alguna de las EDAs.
 - Contar la cantidad de profesores en alguna de las EDAs.
 - vaciar alguna de las EDAs.
 - Crear una torre de desafío, según las reglas del juego.
- (20 puntos) Crear un archivo, llamado `tournament.c`, en donde implemente la interfaz capaz de efectuar todo el proceso de peleas en una torre de desafío. La interfaz debe incluir, al menos, las siguientes funcionalidades:
 - Efectuar una pelea y determinar un ganador, considerando las iteraciones que tomó la pelea.
 - Determinar si se efectua un Hundredality!
 - Mostrar los avances del estudiante en la torre de desafío, y en que nivel se encuentra. Que debe ser utilizado entre peleas.

- Actualizar las recompensas del jugador. Para esto debe usar una Estructura de Datos de su elección, diferente a las de `roster.c` y `towers.c`. Además, debe usar obligatoriamente el `struct nivel_de_mejores` o `stats_t`.
 - Ejecutar re-intentos o `Continue?`, de ser necesario.
 - Generar una pantalla de `GAME OVER`.
 - Generar una pantalla de victoria del Campeón Supremo de RAV Kombat.
- (f) (10 puntos) Crear un archivo, llamado `hiscore.c`, en donde se implementa un sistema de los 5 mejores puntajes en el juego. Esta implementación es libre, puede usar cualquier herramienta que usted estime conveniente y asignar puntaje a su criterio. Sin embargo, esta implementación debe ser persistente entre ejecuciones, por lo tanto debe ser almacenada en un archivo de salida, llamado `hiscore.dat` cuyo formato es libre.
- (g) (20 puntos) Cree un programa, en el archivo `main.c`, que permita jugar **RAV Kombat: Choose your destiny**. Este programa deberá permitir seleccionar un personaje, una torre de desafío, mostrar su avance, determinar si el personaje ha salido victorioso o derrotado, cuantos intentos le tomó y cual puntaje obtuvo. Además, debe cerrar mostrando por pantalla la lista de mejores puntajes históricos obtenidos en todas las ejecuciones del juego. El programa debe funcionar a través de línea de comando, utilizando menús para que el usuario final pueda seleccionar las opciones correspondientes. **Recuerde que debe dar la opción de cerrar el juego y, cuando esta sea seleccionada, actualizar el archivo de salida de puntajes con la información global de todas las ejecuciones.**

2 REGLAS DE ENTREGA Y CONSIDERACIONES GENERALES

- Este trabajo debe realizarse **individualmente**, vale decir, en **grupos de un (1) estudiante**. No se harán excepciones.
- El programa debe ser desarrollado en lenguaje C, y compilado con la versión de gcc disponible en el servidor Aragorn²: gcc 4.8.5.
- No hay un límite máximo de funciones a realizar. Si ud. desea modularizar al máximo, tiene la libertad de hacerlo.
- La tarea debe ser entregada en la plataforma **AULA USM**, el día Viernes 30 de Mayo de 2025 hasta las 23:59:59 Hora de Chile Continental (UTC -4).
- Cada minuto de atraso implicará un descuento siguiendo la sucesión de Fibonacci. Así, diez minutos de atraso implicarían un descuento de cincuenta y cinco (55) puntos ($f(10) = 55$).
- La tarea debe incluirse en un archivo comprimido **.tar.gz**. El nombre del archivo debe seguir la siguiente estructura: **tarea1-eda-nombre-apellido-paralelo.tar.gz**; e.g., **tarea1-eda-oliver-atom-p200.tar.gz**.
- El archivo comprimido debe incluir, al menos, lo siguiente:
 - Cabecera(s) **.h**: Archivo cabecera en el cual se deben incluir todas las bibliotecas a usar en el programa, además de las definiciones de macros, variables globales, tipos de datos personalizados, struct y prototipos de todas las funciones.
 - Códigos **.c**: Código del programa solicitado. Considere que éste es un desafío de al menos tamaño mediano, por lo tanto es recomendable dividir el código en diferentes archivos agrupados por su finalidad.
 - **README**: Archivo de texto plano en el cual se debe incluir: (1) una pequeña reseña del programa, (2) las condiciones de compilación y ejecución, (3) las instrucciones de compilación y ejecución, y (4) la información del creador del sistema.
 - **Makefile**: Archivo de compilación automática del sistema. Una cápsula de video para su confección se encontrará disponible en AULA USM.
- La revisión de los programas se hará utilizando diferentes archivos de tamaño variable. Por lo tanto, debe programar de forma genérica para cualquier cantidad de información de entrada.
- Cada fuga de memoria será penalizada. Utilice **valgrind** para verificar la correcta asignación, uso y liberación de memoria en la ejecución de su programa.
- Si su programa no compila, su nota será automáticamente **un cero (0)**.
- La creatividad en la interfaz del juego será premiada **hasta con 10 puntos extra en la nota final de la tarea**.
- Se realizará un ranking, entre las tareas que hayan sido realizadas de forma completa, respecto a la utilización de memoria entregada por **valgrind**. **Los primeros 10 estudiantes en este ranking, tendrán un bono entre 5 y 15 puntos de acuerdo al lugar obtenido, en la pregunta de manejo de memoria del Certamen 1 de la asignatura.**
- Cualquier atisbo de copia será penalizada con máxima severidad.
- No deje la tarea para último momento. Planifique bien sus tiempos y constrúyala paso a paso.
- Consulte sus dudas, lo más pronto posible, a través de los diversos medios de la asignatura.

²`ssh aragorn.elo.utfsm.cl -l cuentausm`